



UNIVERSITY OF CAMBRIDGE

Acceleration of CFD using CUDA

Gabriel Nilsson

Cavendish Laboratory, Department of Physics, J J Thomson Avenue, Cambridge. CB3 0HE

Abstract

Serial CPU and CUDA GPU solvers for the two-dimensional compressible Euler equations are implemented using the MUSCL-Hancock method with the HLLC Riemann solver and a structure-of-arrays data layout. Both codes are applied to the Mach 1.22 shock helium-bubble problem of Bagabir and Drikakis [1]. The resulting flow features are in qualitative agreement with the reference, and the CPU and GPU solutions agree to within two orders of magnitude of double-precision machine epsilon, confirming the correctness of the GPU implementation. Performance benchmarks on an NVIDIA A30 GPU show speedups ranging from $1.8\times$ at 250×99 cells to $21.3\times$ at 1000×394 cells. The sub-linear scaling of speedup with grid size is attributed to fixed GPU overheads. Larger speedup factors are expected at higher resolutions.

Abridged version. This is a shortened version of a 15-page report submitted for the MPhil in Scientific Computing at the University of Cambridge. Headings marked *[omitted]* show where sections have been removed. Nothing in the remaining text has been reworded. The full report is available on request.

1. Introduction

1.1. CPU and GPU Architectures *[omitted]*

1.2. Impact on Code Design for Compressible Flow Solvers *[omitted]*

1.3. Existing Methods for GPU-Based Euler Solvers *[omitted]*

1.4. Scope of this work

This report describes the implementation and performance comparison of serial CPU and CUDA GPU solvers for the 2D compressible Euler equations, applied to the shock-bubble interaction problem of Bagabir and Drikakis [1]. Section 2 describes the numerical method and implementation choices for both architectures. Sections 3-5 present validation results, performance benchmarks, and a hardware-specific performance analysis. Section 6 draws conclusions and discusses extensions.

2. Methodology

2.1. Governing Equations *[omitted]*

2.2. Numerical Scheme

2.2.1. Choice of Method

The MUSCL-Hancock method [2] was selected as the high-resolution shock-capturing (HRSC) scheme. This method is second-order accurate in both space and time for smooth flows, and reduces to first-order near discontinuities because of its slope-limiting procedure.

The principal reason for choosing MUSCL-Hancock over SLIC and WAF is because of its wide usage in modern CFD codes, such as FLASH and ATHENA. Which in turn stems from it being a robust and second-order-accurate [2, 3].

Additionally, Bagabir and Drikakis use MUSCL reconstruction rather than SLIC or WAF, however with an implicit solver rather than the Hancock predictor-corrector step [1]. Making MUSCL-Hancock a more natural choice for comparison of results.

2.2.2. MUSCL-Hancock algorithm *[omitted]*

2.2.3. HLLC Riemann Solver

HLLC was chosen over HLL because it resolves the contact discontinuity, which is important for accurately

capturing the helium-air interface features in the shock-bubble problem. An exact Riemann solver was not deemed necessary, as HLLC has comparable accuracy at lower computational cost per cell.

2.2.4. Time Integration

The timestep is determined using the formula for dimensionally split schemes as described by Toro [2, Eq. 16.37]:

$$\Delta t = C_{cfl} \times \min_{i,j} \left(\frac{\Delta x_{i,j}}{S_{i,j}^{n,x}}, \frac{\Delta y_{i,j}}{S_{i,j}^{n,y}} \right), \quad (1)$$

where $S_{i,j}^{n,x}$ is the speed of the wave travelling in direction x , at iteration n , in cell (i, j) . A CFL value of 0.9 is selected, as advised in Toro [2, Ch 16.3.2].

The two-dimensional update is performed using Strang splitting, in which the x - and y -sweeps are applied sequentially, in a different order every time-step. Without alternating sweep order, you would introduce a first-order splitting error in time.

2.2.5. Initial Conditions

The initial conditions follow the shock-bubble configuration of Bagabir and Drikakis [1]. The domain is $[0, 0.225] \times [-0.0445, 0.0445]$ m with a default resolution of 500×197 cells. A right-propagating Mach 1.22 shock is initialised at $x=0.005$ m, with the post-shock state determined using the Rankine-Hugoniot jump conditions for a stationary ideal gas [2, Eqs. 3.53-3.56]:

$$\rho_* = \rho_R \frac{(\gamma + 1)M_S^2}{(\gamma - 1)M_S^2 + 2}, \quad (2)$$

$$p_* = p_R \frac{2\gamma M_S^2 - (\gamma - 1)}{\gamma + 1}, \quad (3)$$

$$v_{x,*} = (1 - \frac{\rho_R}{\rho_*})S_3 + \frac{v_{x,R}\rho_R}{\rho_*}, \quad (4)$$

Where $M_S = 1.22$, $\rho_R = 1.29$ kg/m³, $p_R = 1.01325 \times 10^5$ Pa, and $a_R = \sqrt{\gamma p_R / \rho_R}$. The helium bubble is a circle of radius 0.025 m centred at (0.035, 0) m, with $\rho_{He} = 0.214$ kg/m³ and $p = p_R$. Pre-shock air occupies the remainder of the domain.

2.2.6. Boundary Conditions

Transmissive boundary conditions are applied on all four domain boundaries, with two layers of ghost cells ($N_g=2$) as required by the MUSCL-Hancock stencil which accesses cells $i-1$ to $i+1$ during reconstruction. A transmissive boundary condition imposes a zero-gradient condition which allows waves to exit the domain with minimal reflection.

2.3. CPU Implementation

2.3.1. Data Layout

The grid data is stored as four separate one-dimensional arrays, one for each conserved variable (ρ , ρv_x , ρv_y , E), using a structure-of-arrays (SoA) layout. The length of each array is $(N_x + 2N_g)(N_y + 2N_g)$. Elements are stored in row-major order to be contiguous in i , the x -coordinate.

The SoA layout is chosen for the coalesced-access benefits.

2.3.2. Sweep Implementation

Each directional sweep allocates a temporary buffer of primitive-variable pairs (the evolved left and right states) along the relevant one-dimensional strip. For the x -sweep, the outer loop iterates over rows (j) and the inner computation proceeds along columns (i). This ordering ensures that innermost memory accesses are contiguous in the flattened array, favouring CPU cache-line utilisation and hardware prefetching.

For the y -sweep the loop order is reversed: the outer loop runs over columns (i) and the inner loop runs over rows (j). This causes the innermost accesses to stride across rows by $(N_x + 2N_g) \times 8$ bytes. This stride is expected when applying directional sweeps on a grid and cannot be completely eliminated without transposing the data.

2.3.3. Conversion Functions and Inlining

Frequently called physics routines (primitive-to-conservative conversion, sound speed, flux evaluation, HLLC solver), are declared `inline` to decrease function-call overhead.

2.4. GPU Implementation

The GPU implementation mirrors the algorithmic structure of the CPU code but decomposes each stage of the time-integration loop into separate CUDA kernels. All grid data reside in device (global) memory throughout the simulation; host-device transfers occur only during initialisation, output, and Δt -reduction (excluding synchronisations done during performance analysis).

2.4.1. Data Layout and Memory Coalescing

The same SoA, row-major layout is used on the GPU, yielding coalesced memory access across the x -axis (i) and enabling the device to access up to 32 (a warp of threads) values in a single memory transaction. A thread-block size of $B_x=32$ and $B_y=8$ is used throughout, so that the x -dimension exactly matches the warp

size, ensuring full coalescing for every global memory access in the interior compute kernels.

Along the y -axis (j), special care needs to be taken to gain benefits from warp-wide memory transactions, which is discussed in Section 2.4.3.

For the default resolution of 500×197 , with $N_g=2$, having 4 conserved-variable arrays plus 2 Prim arrays for the evolved states `d_eL` and `d_eR` (each with 4 fields), sums to a total of $4 + 2 \times 4 = 12$ arrays of 504×201 doubles, requiring $12 \times 504 \times 201 \times 8 \approx 9.7$ MB of global device storage. Well within the A30's 24 GBs.

2.4.2. Kernel Decomposition [omitted]

2.4.3. Use of Shared Memory

Shared memory is used in the following two kernels.

CFL reduction.

(`min_dt_kernel`)

Each thread loads its local per-cell minimum into shared block memory, and the reduction proceeds in $\log_2(B_x \cdot B_y)$ stages with a `__syncthreads()` call between each stage. This avoids repeated global-memory reads during the reduction and exploits the substantially lower latency of shared memory (~ 20 -30 cycles versus ~ 200 -400 cycles for uncached global load on modern NVIDIA architectures).

y-direction reconstruction.

(`reconstruct_y_kernel`)

The y -sweep stencil reads rows $j-1$, j , and $j+1$. In row-major storage, adjacent rows are $(N_x + 2N_g) \times 8$ bytes apart, far outside any L1 cache line, making every stencil-neighbour access an L2 or DRAM fetch.

To decrease this traffic, the kernel loads a shared-memory tile of shape $(B_y+2) \times B_x$ per conserved field before performing reconstruction. Row 0 in the tile holds the top halo (row $j-1$ for the topmost thread in the block), rows 1 through B_y hold the B_y interior rows owned by the block, and row B_y+1 holds the bottom halo.

The loading protocol is as follows: Every thread loads its own cell into shared row t_y+1 ; the thread with $t_y=0$ additionally loads row $j-1$ into shared row 0; the thread with $t_y=B_y-1$ loads row $j+1$ into shared row B_y+1 (with a bounds check for the last block).

A single `__syncthreads()` barrier after the three load operations ensure all shared data are visible before any thread begins reading its stencil. Threads then read $\mathbf{W}_{j-1}$, $\mathbf{W}_j$, and $\mathbf{W}_{j+1}$ from shared memory rather than global memory.

The shared-memory allocation is $4 \times (B_y+2) \times B_x \times 8 = 10,240$ bytes = 10 KB per block ($B_x=32$, $B_y=8$), well

within the A30's 48 KB per-block limit. Global memory load traffic per block falls from $3 \times B_y \times B_x \times 4$ reads to $(B_y+2) \times B_x \times 4$ reads, a reduction of approximately 58%.

The x -reconstruction kernel does not use shared memory because x -neighbours are 8 bytes apart in row-major storage, placing them in the same or an adjacent cache line; the hardware L1 cache handles this reuse without explicit management.

2.4.4. Thread-Block Configuration

All two-dimensional kernels use a block size of $32 \times 8 = 256$ threads. The x -dimension of 32 matches the warp size, ensuring coalesced global memory access as described above. The y -dimension of 8 provides sufficient parallelism per block to hide some memory latency through warp scheduling, while keeping the total thread count at 256, a value that allows high occupancy on NVIDIA A30 hardware (with 64 warps per SM and 1024 threads per block).

2.4.5. Host-Device Synchronisation and Timing

CUDA events are used to measure the elapsed time for each algorithmic component (boundary conditions, CFL reduction, and sweeps) independently. An event-pair brackets each component, and `cudaEventSynchronize` is called on the trailing event before querying the elapsed time with `cudaEventElapsed`. This provides accurate GPU-side timing without the overhead of a full `cudaDeviceSynchronize` between every kernel launch.

The CFL reduction involves a `cudaMemcpy` from device to host to transfer the per-block minima, which implicitly synchronises the preceding kernel. No other device-to-host synchronisation occurs during the time-stepping loop (excluding VTK output snapshots), allowing the GPU to execute successive kernels asynchronously where possible.

2.4.6. The `HOSTDEV` Macro and Code Sharing

A preprocessor macro `HOSTDEV` is defined to expand to `__device__ __host__` when compiled with NVCC and to an empty token otherwise. All physics functions in the shared header `euler.h` are annotated with this macro. This allows a single implementation of the numerical scheme to be compiled for both host and device targets, eliminating a potential source of inconsistency between the CPU and GPU codes.

2.5. Compilation and Build System

Both codes are compiled with `-O2` optimisation. The CPU code is compiled with GCC using C++17 and

the `-Wall` flag for warnings. The GPU code is compiled with NVCC, also using the C++17 standard, targeting the `sm_80` architecture (Ampere generation, which includes the NVIDIA A30). The `-gencode arch=compute_80,code=sm_80` flags ensures that device code is compiled specifically for this architecture, enabling the compiler to make use of Ampere-specific instruction scheduling and register allocation.

3. Results 1: Simulation of the Mach 1.22 Shock-Bubble Problem

3.1. Setup

Both codes were run at the standard resolution of 500×197 cells with transmissive boundaries on all four sides and a final time of $T = 4.27 \times 10^{-4}$ s. Eight snapshots are written at equal intervals of $\Delta t_{\text{snap}} \approx 61 \mu\text{s}$. All simulation parameters are as listed in Section 2.

3.2. Qualitative Features and Comparison to Literature

Figures 1-4 show numerical schlieren fields $\varphi = \exp(-20|\nabla\rho|/|\nabla\rho|_{\text{max}})$ at $t \approx [61, 183, 305, 427] \mu\text{s}$. Each figure contains the CPU result (top), the GPU result (middle), and the pointwise absolute density difference $|\rho_{\text{CPU}} - \rho_{\text{GPU}}|$ (bottom). The CPU and GPU schlieren fields are visually indistinguishable at this scale, and the pointwise density difference is less than 10^{-13} everywhere.

The sequence captures the principal stages of the shock-bubble interaction: the transmitted shock advancing faster inside the helium bubble due to its higher sound speed ($t \approx 61 \mu\text{s}$), diffracted wave reconnection and bean-shaped interface deformation ($t \approx 183 \mu\text{s}$), and the onset of vortex-ring formation at the downstream cusp ($t \approx 305 - 427 \mu\text{s}$).

These features are qualitatively consistent with the schlieren images reported by Bagabir and Drikakis [1] for the same Mach number. This work simulates the entire bubble at 500×197 cells, whereas the reference simulates only the upper half of the domain. Furthermore, the precise initial shock position used in the reference is not specified [1], so exact timing agreement is not expected. Differences in feature sharpness relative to the reference at this resolution are expected because of the `minmod` limiter, which is more diffusive than the van Leer limiter used by Bagabir and Drikakis.

In Figure 5 a density contour plot from Bagabir and Drikakis is replicated [1, Fig. 4], and the two are compared next to each other in Figure 6.

Comparing the two we start seeing clear discrepancies at $t = 476 \mu\text{s}$, and at $t = 772 \mu\text{s}$ we see that the

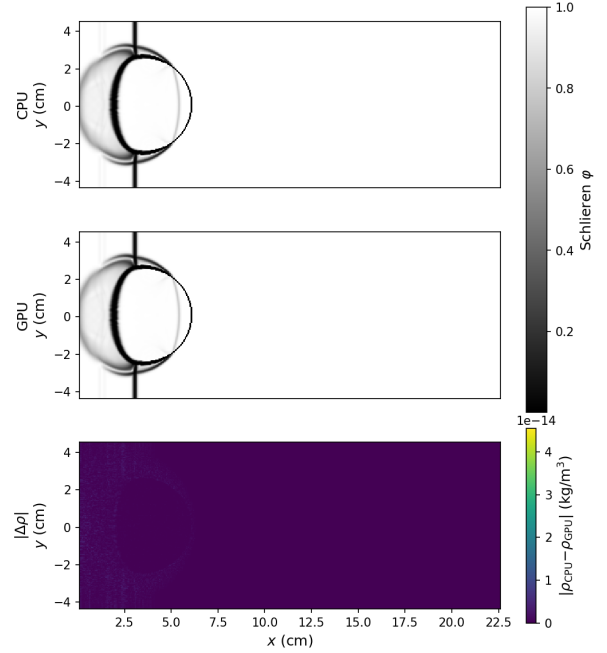


Figure 1: $t \approx 61 \mu\text{s}$. Top: CPU schlieren. Middle: GPU schlieren. Bottom: density difference $|\rho_{\text{CPU}} - \rho_{\text{GPU}}|$ (kg/m^3).

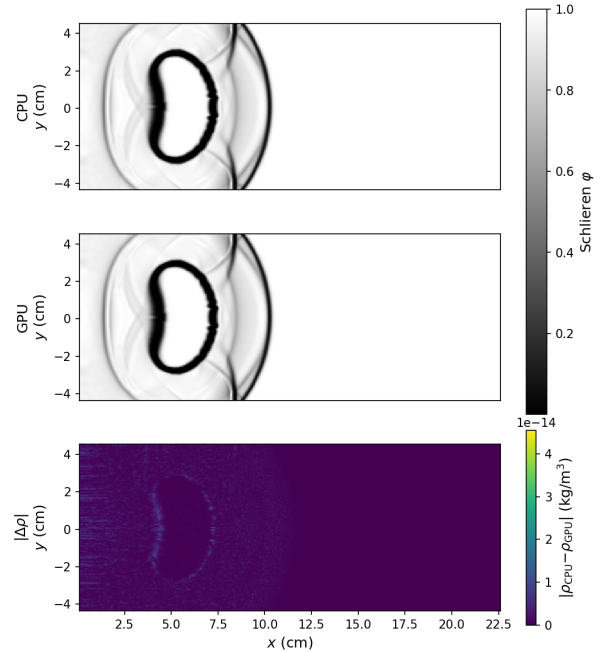


Figure 2: $t \approx 183 \mu\text{s}$. Layout as Figure 1

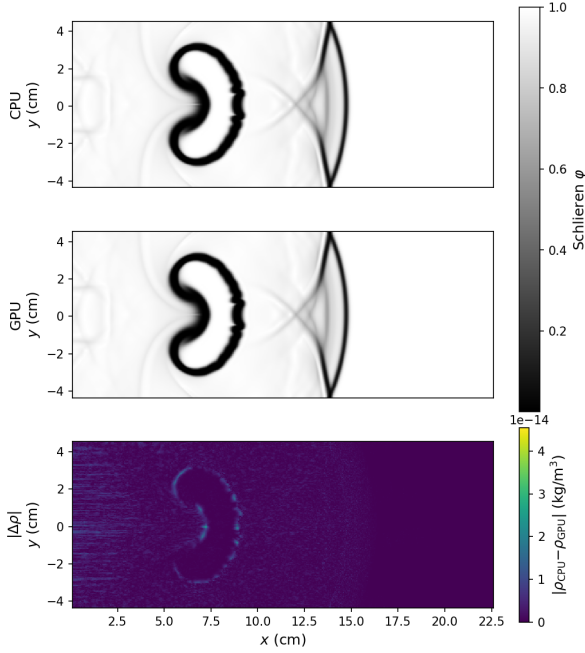


Figure 3: $t \approx 305 \mu\text{s}$. Layout as Figure 1

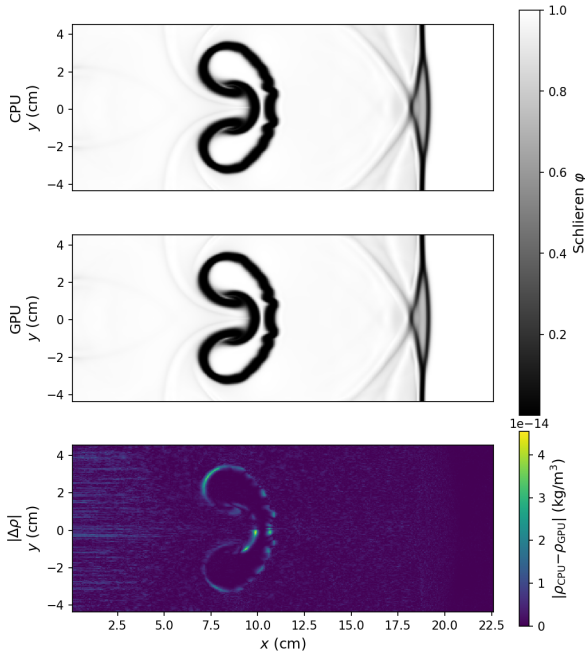


Figure 4: $t \approx 427 \mu\text{s}$. Layout as Figure 1

front of the helium bubble is further downstream in my implementation than in [1, Fig. 4].

This discrepancy can partly be explained by the fact that the start-location for the shock is not defined in [1], making alignment in time imperfect. In addition, this work uses the minmod slope limiter, which is more diffusive than the van Leer limiter used in the reference.

Furthermore, the boundary conditions differ: Bagabir and Drikakis simulate only the upper half of the domain with a reflective boundary along the centreline. This work simulates the entire bubble with transmissive boundaries on all four sides. Grid-scale asymmetries could hence subtly alter the bubble deformation at late times.

Finally, Bagabir and Drikakis use an implicit time-integration scheme, which may exhibit different numerical diffusion characteristics. Collectively these differences in implementation could reasonably explain the discrepancies observed in Figure 6, especially at later times when small differences in wave propagation speed have accumulated.

3.3. CPU-GPU Agreement

The bottom panels in Figures 1-4 show the pointwise absolute density difference $|\rho_{\text{CPU}} - \rho_{\text{GPU}}|$. The discrepancies are confined to shock and interface locations, where the two codes differ by at most $\approx 5 \times 10^{-14} \text{ kg/m}^3$.

Table 1 summarises the L^2 relative error and maximum absolute error in density between the two codes. We find that the absolute error is within two orders of magnitude of the double-precision machine epsilon at $\approx 2 \times 10^{-16}$, and that the relative error is negligible.

Snapshot	$t \text{ (}\mu\text{s)}$	ε_{L^2}	$\varepsilon_{\infty} \text{ (kg/m}^3\text{)}$
1	61	3.11×10^{-16}	4.88×10^{-15}
3	183	7.21×10^{-16}	1.22×10^{-14}
5	305	1.09×10^{-15}	2.25×10^{-14}
7	427	1.63×10^{-15}	4.55×10^{-14}

Table 1: CPU-GPU density agreement at four selected times. $\varepsilon_{L^2} = \|\rho_{\text{CPU}} - \rho_{\text{GPU}}\|_2 / \|\rho_{\text{CPU}}\|_2$; $\varepsilon_{\infty} = \max |\rho_{\text{CPU}} - \rho_{\text{GPU}}|$.

4. Results 2: CPU vs GPU Performance

Both solvers were benchmarked at four grid resolutions, from 250×99 ($\approx 25\,000$ cells) up to 1000×394 ($\approx 394\,000$ cells), running 500 time steps each to obtain stable per-step averages. All CPU timings were measured on a single core of a modern x86 processor; all GPU timings on the NVIDIA A30 on the Lovelace node. The wall-time of the three instrumental

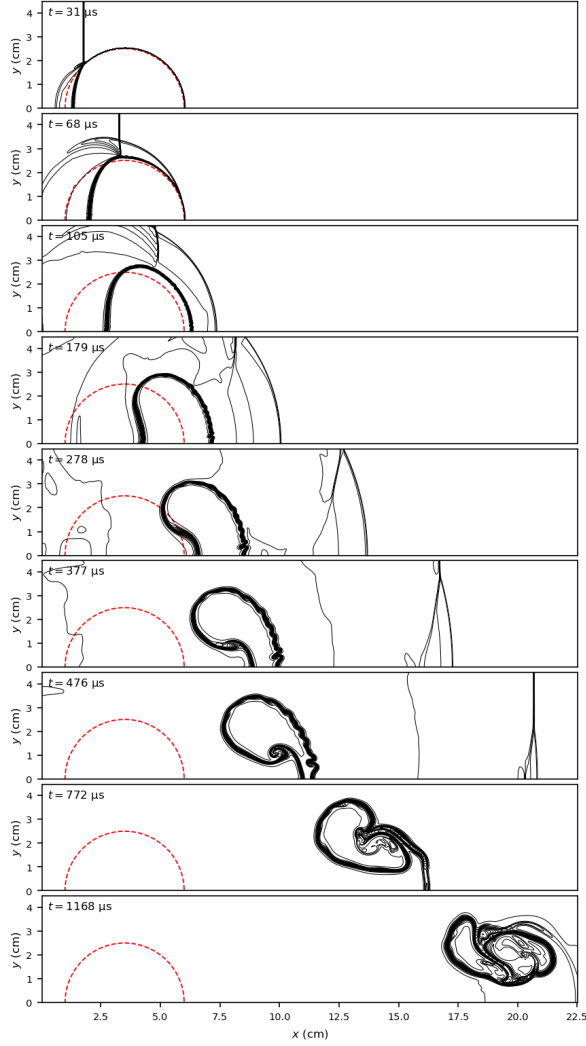


Figure 5: Density contour for shock-bubble interaction at $M_S = 1.22$. Red circle highlights initial position of helium bubble.

components (BC, Δt -reduction, and MUSCL-Hancock sweeps) are tracked separately to identify performance bottlenecks.

Component breakdown.

Tables 2 and 3 summarise the time per step for each component (BC, Δt -reduction, and sweeps) and wall time. To access the per-component wall-time, two `cudaEventSynchronize` calls were required. Since these synchronisations require host-device communication they meaningfully increase the overall wall-time. The simulation is only run with these synchronisations when getting the per-component time-values. Omitting them when getting the total wall-time.

On the CPU the MUSCL-Hancock sweeps account

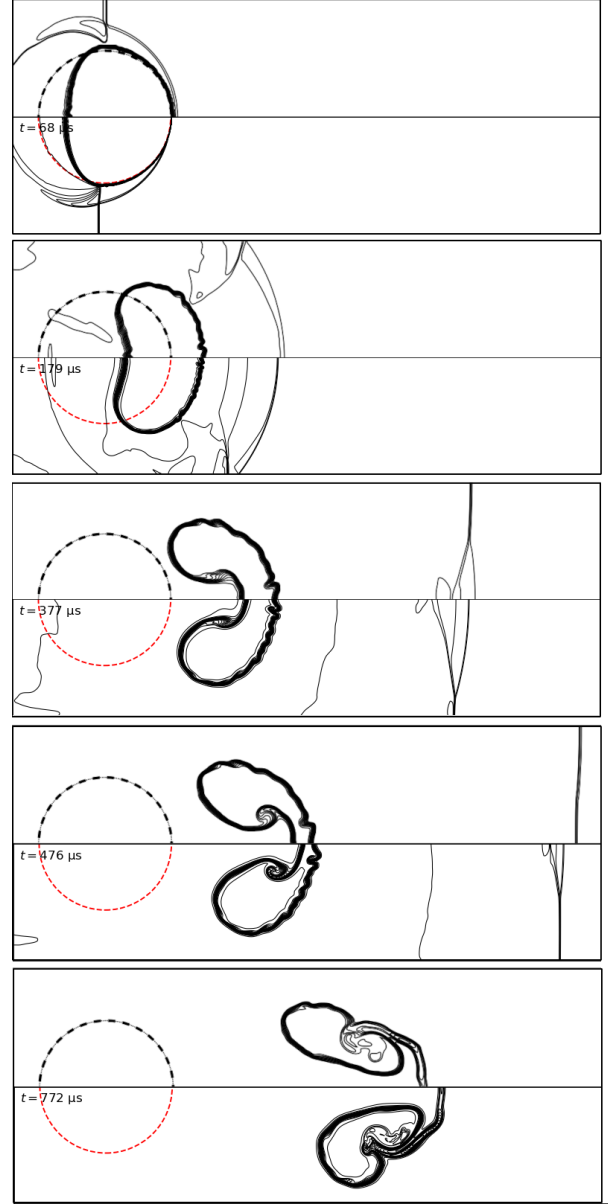


Figure 6: Density contour for shock-bubble interaction at $M_S = 1.22$. Top half is from Bagabir and Drikakis [1, Fig. 4]; slices 2, 4, 6, 7, and 8 respectively. Lower half is this report's GPU implementation. Red and black circle highlights initial position of helium bubble. Note that the initial shock-position is not defined by Bagabir and Drikakis, making alignment between results imperfect.

for $\approx 94.5\%$ of the total wall time at every resolution, with the CFL reduction contributing the remaining $\approx 5.5\%$ and boundary conditions $\approx 0.1\%$. This confirms that the flux computation is responsible for the vast majority of the computational work, and that any optimisation effort should target the sweep kernels.

$NX \times NY$	BC	Δt	Sweep	Wall time
250 $\times$ 99	0.005 <i>0.1%</i>	0.324 <i>5.4%</i>	5.672 <i>94.5%</i>	6.001 <i>100%</i>
500 $\times$ 197	0.032 <i>0.1%</i>	1.300 <i>5.4%</i>	22.693 <i>94.5%</i>	24.025 <i>100%</i>
750 $\times$ 296	0.055 <i>0.1%</i>	2.347 <i>5.4%</i>	41.330 <i>94.5%</i>	43.732 <i>100%</i>
1000 $\times$ 394	0.098 <i>0.1%</i>	4.726 <i>5.5%</i>	81.237 <i>94.4%</i>	86.061 <i>100%</i>

Table 2: CPU wall time per step at four resolutions (500-step averages). Each resolution shows two rows: absolute time in milliseconds (plain) and percentage of the total step time (*italic*). Component times are measured with `std::chrono`.

On the GPU, note that the sum of component percentages is not 100%, but between 78–91% (Table 3). This is because CUDA event timers do not measure the time between the kernel launches where the GPU is idle. However, the wall-time includes everything, including CPU-GPU synchronisation.

$NX \times NY$	BC	Δt	Sweep	Wall time
250 $\times$ 99	0.029 <i>0.8%</i>	0.027 <i>0.8%</i>	2.974 <i>87.5%</i>	3.398 <i>100%</i>
500 $\times$ 197	0.028 <i>0.7%</i>	0.036 <i>1.0%</i>	2.820 <i>76.5%</i>	3.969 <i>100%</i>
750 $\times$ 296	0.029 <i>0.8%</i>	0.052 <i>1.4%</i>	3.259 <i>89.2%</i>	3.655 <i>100%</i>
1000 $\times$ 394	0.030 <i>0.8%</i>	0.071 <i>1.8%</i>	3.254 <i>80.4%</i>	4.045 <i>100%</i>

Table 3: GPU wall time per step at four resolutions (500-step averages). Each resolution shows two rows: absolute time in milliseconds (plain) and percentage of the total step time (*italic*). Component times are measured with CUDA events. GPU component percentages sum to only 78–91% of wall time because the remainder is CPU-GPU synchronisation latency, e.g. `cudaMemcpy` in the CFL reduction, which is not captured by the CUDA events.

For each resolution, the time unaccounted for (wall time – (BC + Δt + sweep)) is: 0.424, 1.149, 0.396, and 0.791 ms; which respectively is 10.8%, 21.8%, 8.6% and 17.1% of the wall time. This time is spent on kernel launch overhead and host-device traffic like the `cudaMemcpy` used to retrieve the CFL value.

The inconsistency across resolutions is expected because the benchmarks were run on a shared node

with non-zero occupancy from other users. CUDA event times measure GPU-active execution and are relatively stable, but wall-clock measurements include OS scheduling latency and PCIe bus contention from concurrent processes, both of which fluctuate between runs.

Despite the larger fraction of wall-time spent on synchronisation (Figure 7), significant overall speedups are being achieved, which scale from 1.8 $\times$ to 21.3 $\times$ as we go through the grid resolutions (Figure 8 & Figure 9).

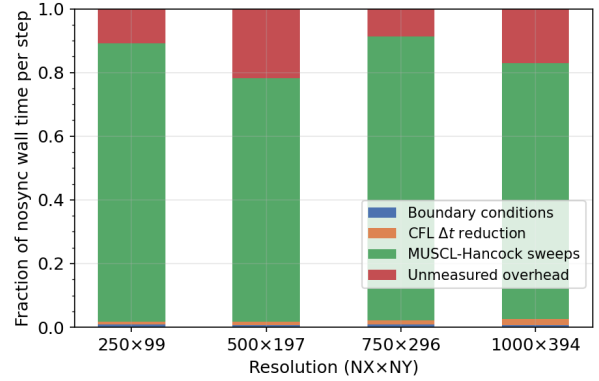


Figure 7: Fraction of wall-time spent being in the three different phases of the algorithm, and doing GPU-CPU synchronisation, across the four different grid resolutions; for the GPU implementation.

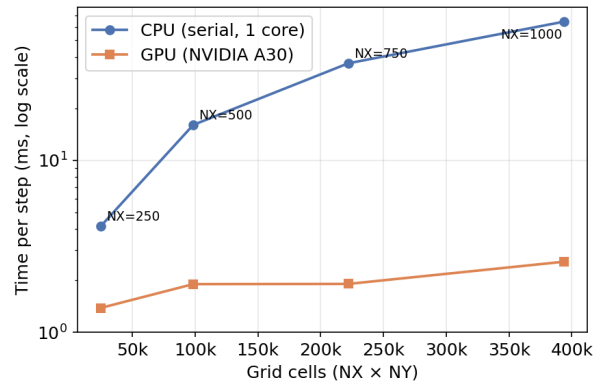


Figure 8: Wall time per step over number of grid cells; for both serial CPU implementation and GPU implementation. Hovering labels show which of the four grid resolutions this cell count is associated with.

5. Results 3: Hardware-Specific Performance Analysis

5.1. Roofline Analysis and Memory-Boundedness

The performance characteristics observed in Section 4 can be explained by analysing the algorithm-

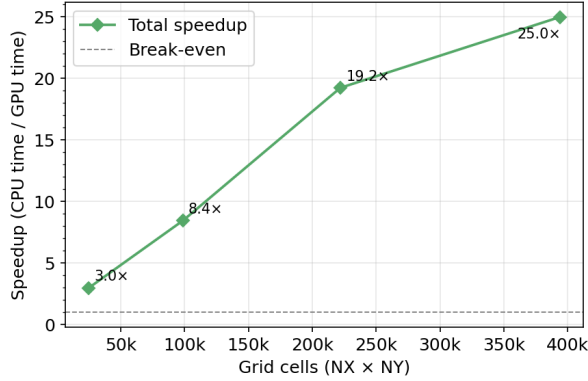


Figure 9: How many times faster the GPU implementation runs compared with the CPU implementation, over the number of grid cells. Dashed line shows where the GPU implementation would be as fast as the CPU implementation. The slope of best fit corresponds with a $\approx 6.1\times$ speedup per 100k cells.

mic workload against the hardware capabilities of the NVIDIA A30.

The A30’s roofline ridge point for double-precision arithmetic is 5.2 TFLOPS/933 GB/s ≈ 5.6 FLOPs/byte [4]. Since the arithmetic intensity of the MUSCL-Hancock sweep (≈ 0.8 FLOPs/byte) falls well below this ridge point, the sweep kernels are memory-bandwidth-bound, i.e. performance is limited by the rate at which data can be moved to and from HBM2, not by the floating-point throughput of the CUDA cores.

5.2. GPU Utilisation and Scaling with Grid Size

At the grid sizes benchmarked, the A30 is significantly underutilised. The GPU sweep kernel time is nearly constant at ≈ 3 ms across all four resolutions (Table 3), varying by only $1.16\times$ despite a $15.9\times$ increase in cell count from 250×99 to 1000×394 . This indicates that the smaller grids do not generate enough concurrent thread blocks to keep all 56 streaming multiprocessors busy, and that even at the largest grid ($\approx 394\,000$ cells) the achieved memory bandwidth remains a small fraction of the A30’s peak. The GPU is spending most of its time on fixed-cost overheads, such as kernel launch latency, the `cudaMemcpy` in the CFL reduction, and implicit synchronisations.

As a result, the observed speedup grows steadily with resolution, from $1.8\times$ to $21.3\times$ as can be seen in Figure 9. This sub-linear scaling relative to the theoretical bandwidth advantage is characteristic of a regime where fixed GPU overheads dominate. Since the CPU wall time scales linearly with cell count ($O(N)$) while the GPU wall time remains approximately flat, the speedup

ratio grows roughly in proportion to N . The GPU would be expected to reach a plateau speedup at grids much larger than those tested, once the computation time exceeds the fixed overhead and memory bandwidth becomes the bottleneck.

5.3. Impact of Implementation Choices

Structure-of-arrays data layout.

The SoA layout ensures coalesced GPU memory access (Section 2.4), allowing a warp’s reads to be serviced in a single wide transaction. On the CPU, the same layout produces a sequential access pattern that the hardware prefetcher can exploit to keep L1/L2 cache lines filled ahead of demand.

Shared-memory tiling in the y -reconstruction.

The shared-memory halo strategy in the y -reconstruction (Section 2.4.3) reduces global memory loads per block by $\approx 58\%$, addressing the memory-bound bottleneck identified by the roofline analysis. The x -reconstruction relies on L1 cache reuse, since x -neighbours are 8 B apart.

Thread-block configuration and occupancy.

The $32\times 8=256$ thread block matches the warp size in the x -direction. Making threads within a warp always access adjacent memory locations. The A30 supports up to 2048 threads per SM, giving a theoretical maximum of $2048/256 = 8$ blocks per SM. However, the different kernels use up some of the register budget during run time. Static resource usage analysis (dynamic resource usage is unreliable as the GPU is communal and also running other tasks) of the compiled CUDA binary shows that the `update_x/y_kernel` theoretically use 68 registers per thread, `reconstruct_x/y_kernel` use 48, the other kernels using less than 35.

As the A30 has 65 536 registers per SM, with 256 threads per block, one block needs $68\times 256=17\,408$ registers during the update kernel (HLLC), meaning that the SM can fit at most $\lfloor 65536/17408 \rfloor = 3$ blocks simultaneously, making use of 24 warps per SM out of a maximum of 64, an occupancy of $\approx 37\%$. The reconstruction kernels at 48 registers would allow about 5 blocks, 40 warps out of 64, giving $\approx 62\%$ occupancy. 40 warps is a reasonable number for the SM scheduler to overlap memory transactions with computation from other warps, however since the arithmetic intensity of the MUSCL-Hancock is low (≈ 0.8 FLOPs/byte), even 40 warps may not be enough to keep the SM continuously occupied.

Host-device synchronisation overhead.

As shown in Table 3, the time unaccounted for by CUDA events constitutes 9–22% of wall time, staying approximately constant across resolutions, just like the sweep kernel time. Performing the full CFL reduction on-device via a second-stage reduction kernel would eliminate the per-step `cudaMemcpy` and reduce this overhead.

HOSTDEV code sharing.

The `HOSTDEV` macro makes all physics functions compiled identically for both host and device targets, eliminating a class of bugs that could arise from maintaining separate CPU and GPU implementations. This way near-machine-epsilon agreement is achieved, as observed in Table 1. It also means that the NVCC compiler can inline these functions into the device kernels, reducing function-call overhead and enabling register-level optimisation of the reconstruction-update pipeline within each thread.

6. Conclusions

A serial CPU and CUDA GPU solver for the 2D compressible Euler equations were implemented using the MUSCL-Hancock method with HLLC fluxes and a structure-of-arrays data layout. Both codes produce results consistent with the Bagabir and Drikakis shock-bubble benchmark, and agree within each other to within two orders of magnitude of double-precision machine epsilon, confirming the correctness of the GPU implementation.

Speedups of $1.8\times$ to $21.3\times$ were observed across all resolutions. A roofline analysis showed that the sweep kernels are memory-bandwidth-bound, with an arithmetic intensity of ≈ 0.8 FLOPs/byte, well below the A30's ridge point of 5.6 FLOPs/byte. The grids tested are too small to saturate the A30's 933 GB/s memory bandwidth, meaning substantially larger speedups are expected at higher resolutions where the fixed synchronisation overhead becomes a smaller fraction of wall time.

Two concrete extensions to the current implementation would improve performance. First completing the CFL reduction entirely on-device would decrease the latency associated with host-device communication, which currently is a non-negligible fraction of wall time. Second, extending the shared-memory tiling strategy used in the y -reconstruction to the update kernels would further reduce global memory traffic in the memory-bound regime.

Appendix A. High Resolution Schlieren Plots [omitted]

References

- [1] A. Bagabir, D. Drikakis, Mach number effects on shock-bubble interaction, *Shock Waves* 11 (3) (2001) 209–218.
- [2] E. F. Toro, *Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction*, 3rd Edition, Springer, 2009.
- [3] K. Murawski, K. Murawski, P. Stpiczyński, Implementation of MUSCL-Hancock method into the C++ code for the Euler equations, *Bulletin of the Polish Academy of Sciences: Technical Sciences* 60 (1) (2012) 45–53. doi:10.2478/v10175-012-0008-7.
- [4] NVIDIA Corporation, NVIDIA A30 Tensor Core GPU Datasheet, available at <https://www.nvidia.com/en-us/data-center/products/a30-gpu/> (2021).